

PracticeGrading: техническая документация

Проект: сопровождение и развитие PracticeGrading

Репозиторий: <https://github.com/yurii-litvinov/PracticeGrading>

1. Назначение документа

Документ описывает реализованные средства управления администраторами и доступом членов комиссии, соответствующие элементы веб-интерфейса, а также подготовленный процесс Continuous Deployment (CD). Отдельно перечислены файлы, каталоги и права, необходимые на production-сервере.

2. Общая архитектура

PracticeGrading состоит из следующих частей:

Компонент	Технологии	Назначение
Frontend API	React, TypeScript, Vite, Bootstrap ASP.NET Core 8 Minimal API	Интерфейс администратора и членов комиссии Бизнес-логика, авторизация, выдача токенов, обработка оценок
Хранилище	PostgreSQL	Пользователи, заседания, оценки, заявки и доверенные доступы
Обновления в реальном времени	SignalR	Синхронизация действий участников заседания
Reverse проху	Nginx	HTTPS, публикация frontend и проксирование API
Сборка и доставка	GitHub Actions, GHCR, self-hosted runner	Проверка, упаковка и развёртывание релиза
Запуск приложения	Docker Compose	API и PostgreSQL на production-сервере

3. Аутентификация и виды токенов

В системе применяются три разных токена. Их нельзя смешивать или сохранять под одним ключом.

Токен	Для чего нужен	Где находится у клиента	Что хранится в БД	Как прекращается доступ
JWT администратора или члена комиссии	Авторизованные API-запросы	sessionStorage, ключ token	Сам JWT не хранится	Истечение JWT; для члена комиссии дополнительно проверяется активность связанного доступа
Токен заявки на заседание	Проверка статуса одной заявки и чтение сведений о конкретном заседании	localStorage, ключ meeting-member-access-token:<meetingId>	SHA-256-хеш токена	Отклонение или отзыв заявки
Доверенный токен	Вход доверенного члена комиссии в разные заседания без ручного подтверждения каждой заявки	localStorage, ключ trusted-member-access-token	SHA-256-хеш токена	Отзыв доверенного доступа или перевыпуск ссылки

Секретные токены возвращаются API в исходном виде только при создании. В базе сохраняются их хеши. Перевыпуск доверенной ссылки заменяет хеш и делает старую ссылку недействительной.

Политики API:

- RequireAdminRole — только администратор;
- RequireMemberRole — член комиссии с действующим доступом;
- RequireAdminOrMemberRole — администратор либо член комиссии с действующим доступом.

Для JWT члена комиссии обработчик ActiveMemberAccessHandler при каждом защищённом запросе проверяет, что соответствующая заявка или доверенный доступ всё ещё активны. Поэтому отзыв доступа действует и на ранее выданный JWT.

4. API и соответствующий frontend

4.1. Учётные записи администраторов

Метод и маршрут	Доступ	Назначение	Frontend
POST /login	Без авторизации	Вход администратора, возврат JWT	LoginPage, функция loginAdmin
PUT /users/me/password	Администратор	Изменение собственного пароля	ProfilePage, форма «Изменение пароля»
POST /admins	Администратор	Создание нового администратора после повторного ввода текущего пароля	ProfilePage, форма «Создание администратора»

Запрос на смену пароля:

```
{
  "currentPassword": "current-password",
  "newPassword": "new-password-with-12-chars"
}
```

Успешный ответ — 204 No Content. Новый пароль должен содержать не менее 12 символов и отличаться от текущего.

Запрос на создание администратора:

```
{
  "userName": "new-admin",
  "password": "new-password-with-12-chars",
  "currentPassword": "creator-current-password"
}
```

Успешный ответ — 201 Created; 400 означает ошибочные данные или текущий пароль, 409 — занятое имя пользователя. Повторный ввод текущего пароля не позволяет создать администратора только за счёт оставленной открытой сессии.

4.2. Обычный доступ к отдельному заседанию

Метод и маршрут	Доступ	Назначение	Frontend
POST /meetings/{meetingId}/access-requests	Без авторизации	Создать заявку по существующему участнику или введённому ФИО	MemberLoginPage
GET /meetings/{meetingId}/access-requests/status	Заголовок X-Meeting-Access-Token	Получить состояние заявки	Автоматический опрос в MemberLoginPage каждые 3 секунды
GET /meetings/{meetingId}/read-only	Заголовок X-Meeting-Access-Token	Читать сведения о заседании, пока заявка ожидает решения или одобрена	ReadOnlyMeetingPanel
POST /meetings/{meetingId}/member-login	Заголовок X-Meeting-Access-Token	Получить JWT после одобрения	MemberLoginPage
GET /meetings/{meetingId}/access-requests/pending	Администратор или член этой комиссии	Получить ожидающие заявки	MeetingAccessRequestPanel
POST /meetings/{meetingId}/access-requests/{accessId}/approve	Администратор или член этой комиссии	Одобрить заявку	MeetingAccessRequestPanel
POST /meetings/{meetingId}/access-requests/{accessId}/reject	Администратор или член этой комиссии	Отклонить заявку	MeetingAccessRequestPanel
GET /meetings/{meetingId}/access-requests/approved	Администратор	Получить активные одобренные доступы	ApprovedMeetingAccessesPanel
POST /meetings/{meetingId}/access-requests/{accessId}/revoke	Администратор	Отозвать ранее одобренный доступ	ApprovedMeetingAccessesPanel

Создание заявки для существующего участника:

```
{
  "memberId": 42,
  "userName": null
}
```

Если пользователя ещё нет в списке, frontend передаёт memberId: 0 и введённое userName. API возвращает личный token заявки:

```
{
  "token": "secret-meeting-access-token"
}
```

Состояния заявки: Pending (0), Approved (1), Rejected (2), Revoked (3). При Approved frontend автоматически обменивает токен заявки на JWT и открывает страницу члена комиссии. При Rejected или Revoked локальный токен удаляется.

Член комиссии может одобрять и отклонять заявки только для заседания, идентификатор которого записан в его JWT. Администратор может управлять заявками любого заседания.

4.3. Доверенный доступ

Доверенный доступ выдаётся администратором конкретному пользователю с ролью члена комиссии. Полученная персональная ссылка один раз открывается в нужном браузере; после этого общие ссылки на заседания можно открывать без ожидания подтверждения.

Метод и маршрут	Доступ	Назначение	Frontend
GET /members/{memberId}/trusted-access	Администратор	Проверить, был ли доступ выдан и активен ли он	MemberModal
POST /members/{memberId}/trusted-access	Администратор	Выдать или перевыпустить персональный токен	MemberModal
DELETE /members/{memberId}/trusted-access	Администратор	Отозвать доверенный доступ	MemberModal
POST /meetings/{meetingId}/trusted-login	Заголовок X-Trusted-Access-Token	Проверить доверенный токен и вернуть JWT для заседания	MemberLoginPage

MemberModal строит ссылку следующего вида:

`https://<host>/practice-grading/trusted-access#token=<secret>`

Токен расположен после #, поэтому браузер не отправляет его серверу как часть URL. Страница TrustedAccessActivationPage сохраняет токен в localStorage, затем убирает его из адресной строки. При открытии общей ссылки на заседание MemberLoginPage сначала пытается выполнить доверенный вход. Если токен действителен, API при необходимости добавляет пользователя в состав заседания и выдаёт JWT. Если токен отозван, он удаляется из браузера, после чего доступ можно запросить обычным способом.

4.4. Модель данных доступа

MeetingMemberAccess хранит заявку на одно заседание:

- идентификаторы заседания и участника;
- SHA-256-хеш секретного токена;
- состояние заявки;
- время создания и последнего изменения;
- пользователя, обработавшего заявку.

TrustedMemberAccess хранит постоянный доступ участника:

- идентификатор участника;
- SHA-256-хеш токена;
- время выдачи;
- время отзыва либо NULL для активного доступа.

Изменения схемы оформлены SQL-миграцией 001_member_access_authentication.sql. История применённых миграций записывается в таблицу __SchemaMigrations с номером, именем, контрольной суммой и временем применения.

4.5. Проверка работоспособности

GET /health/live не требует авторизации и возвращает 204 No Content, когда процесс API способен принимать запросы. Этот маршрут используется CD после запуска контейнера.

5. Frontend: маршруты и сценарии

Базовый путь frontend — /practice-grading, а production-сборка обращается к API через /practice-grading/api.

Маршрут frontend	Компонент	Сценарий
/login	LoginPage	Вход администратора
/profile	ProfilePage	Смена пароля и создание администратора
/members	Members, MemberModal	Управление членами комиссии и доверенными ссылками
/trusted-access	TrustedAccessActivationPage	Активация персональной доверенной ссылки
/meetings/{id}/member/login	MemberLoginPage	Доверенный вход либо создание и ожидание заявки
/meetings/{id}	ViewMeetingPage	Администратор: заявки, активные доступы и заседание
/meetings/{id}/member	MemberPage	Работа члена комиссии; одобрение ожидающих заявок
/meetings/{id}/studentwork/{workId}	StudentWorkPage	Выставление оценок по критериям

Frontend хранит JWT в `sessionStorage`, поэтому новая вкладка того же сеанса может его использовать, но после завершения сессии браузера он исчезает. Токены заявок и доверенный токен хранятся в `localStorage`, чтобы переживать перезапуск браузера. При ответе 401 или при связанном с заседанием 403 `interceptor` удаляет JWT и перенаправляет пользователя на подходящую страницу входа.

6. Тестирование

Backend проверяется проектом `PracticeGrading.Tests`. Для новой функциональности добавлены проверки endpoint-ов, сервисов и правил авторизации в файлах:

- `EndpointsTests/UserEndpointsTests.cs`;
- `EndpointsTests/MeetingMemberAccessEndpointsTests.cs`;
- `EndpointsTests/HealthEndpointsTests.cs`;
- `ServicesTests/UserServiceTests.cs`;
- `ServicesTests/MeetingMemberAccessServiceTests.cs`;
- `ServicesTests/TrustedMemberAccessServiceTests.cs`;
- `ServicesTests/JwtServiceTests.cs`.

В `frontend/tests/access-and-admin.spec.ts` добавлены сквозные Playwright-сценарии:

1. обычная заявка, одобрение, выставление оценки и отзыв доступа;
2. доверенный вход участника, которого ещё нет в заседании, и блокировка после отзыва;
3. создание нового администратора, вход под ним и изменение пароля.

CD запускает `dotnet build`, `dotnet test`, `dotnet format`, поднимает тестовый Docker Compose, устанавливает зависимости frontend и выполняет полный набор Playwright-тестов.

7. Continuous Deployment

7.1. Условия запуска

Workflow `.github/workflows/ci-cd.yml` запускается на событие `push`.

- Для любой ветки выполняется только сборка и тестирование.
- Упаковка и production-деплой выполняются только при `github.ref == 'refs/heads/main'`.
- Слияние pull request в main создаёт push в main и запускает полный процесс.
- Событие `pull_request` в workflow отсутствует, поэтому код из внешнего pull request не запускается на production runner до слияния.

7.2. Этапы workflow

Job	Runner	Выполняемые действия
build-and-test	GitHub-hosted ubuntu-latest	Backend build/tests/format, Docker Compose, frontend dependencies, Playwright
package-production	GitHub-hosted ubuntu-latest	Сборка и публикация API-образа, сборка frontend, создание release artifact
deploy-production	self-hosted, Linux, practicegrading-production	Загрузка artifact в защищённый входной каталог и запуск root-скрипта через ограниченный sudo

API-образ публикуется в GitHub Container Registry:

`ghcr.io/yurii-litvinov/practicegrading-api:<полный Git SHA>`

Тегом является полный SHA коммита. Он связывает образ с исходным кодом релиза. Сам тег технически может быть перезаписан, поэтому для строгой идентификации образа нужен digest. Для публикации используется временный GITHUB_TOKEN job-a с packages: write; постоянный токен в Secrets для сборки не требуется.

Frontend собирается с:

VITE_API_URL=/practice-grading/api

Release artifact practicegrading-release-<SHA> хранится GitHub Actions 7 дней и содержит:

frontend/	собранные статические файлы
migrations/	SQL-миграции
release-sha	полный SHA релиза
api-image	точное имя и тег API-образа

Ограничение concurrency не позволяет двум production-деплойм выполняться одновременно, а timeout-minutes: 20 останавливает зависший job.

7.3. Поток релиза

```
push
-> build-and-test
-> если main: package-production
    -> API image в GHCR
    -> frontend + migrations + metadata в Actions artifact
-> self-hosted runner скачивает artifact
-> /var/lib/practicegrading-production/incoming
-> sudo /usr/local/sbin/practicegrading-deploy
-> API container + frontend symlink + database migrations
```

Self-hosted runner не выполняет произвольные административные команды через sudo: ему разрешён только один точный вызов /usr/local/sbin/practicegrading-deploy. Сам скрипт и защищённые конфигурации принадлежат root и недоступны для изменения runner-y.

8. Production: каталоги, файлы и права

Путь	Владелец и режим	Содержимое / назначение
/opt/actions-runner/practicegrading-production	системный пользователь practicegrading-runner	Установка GitHub Actions runner и его рабочие файлы
/etc/practicegrading/	root:root, каталог 0700	Защищённые конфигурации deployment и Compose
/etc/practicegrading/deployment.env	root:root, 0600	Пути, имена сервисов, health URL и параметры deploy-скрипта
/etc/practicegrading/production.env	root:root, 0600	Production-переменные Compose, включая секретные значения
/etc/practicegrading/docker-compose.production.yml	root:root, 0600	Проверенная production-конфигурация Docker Compose
/usr/local/lib/practicegrading/	root:root, скрипты 0755	backup-database.sh и run-migrations.sh
/usr/local/sbin/practicegrading-deploy	root:root, 0755	Основной доверенный deploy-скрипт
/etc/sudoers.d/practicegrading-production-deploy	root:root, 0440	Разрешение runner-y запускать только deploy-скрипт
/var/lib/practicegrading-production/incoming	runner, 0700	Временный входной artifact текущего workflow
/var/lib/practicegrading-production/releases/<SHA>/frontend	root:root, каталоги 0755, файлы 0644	Неизменяемые frontend-релизы
/var/lib/practicegrading-production/releases/<SHA>/migrations	root:root, каталоги 0755, файлы 0644	Миграции конкретного релиза
/var/lib/practicegrading-production/current-release	root:root, 0600	SHA успешно активированного релиза
/var/lib/practicegrading-production/maintenance.flag	root	Наличие файла включает ответ Nginx 503 на время обновления
/var/www/practice-grading	символическая ссылка	Указывает на frontend активного релиза
/var/backups/practicegrading/production	root:root, 0700	Проверочный дамп БД перед каждым изменяющим деплоем
/etc/nginx/snippets/practicegrading-production-maintenance.conf	root:root, 0644	Фрагмент Nginx для режима обслуживания

current-release создаётся deploy-скриптом после успешных миграций, запуска API, health check и переключения frontend. При следующем CD файл атомарно перезаписывается новым SHA. Он хранится постоянно между workflow и позволяет распознать уже установленный релиз.

Преддеплойные дампы хранятся отдельно от ежедневных копий. Ежедневное копирование уже настроено, но все копии пока находятся на том же сервере. Потеря сервера или диска требует внешней копии и заранее проверенной процедуры восстановления.

9. Production-конфигурация

9.1. production.env

Файл используется Docker Compose для передачи переменных контейнерам и подстановки API_IMAGE. В нём должны быть реальные production-значения, перенесённые из действующей конфигурации без публикации в Git:

- API_IMAGE — точный образ API;
- HOST — production-домен;
- параметры подключения PostgreSQL;
- JWT issuer, audience и secret либо соответствующая конфигурация приложения;
- иные уже используемые production-переменные.

Deploy-скрипт изменяет только строку API_IMAGE= и делает это атомарно. Секреты не должны попадать в workflow artifact, логи, репозиторий или каталог runner-a.

9.2. Защищённый Docker Compose

Окончательный /etc/practicegrading/docker-compose.production.yml создаётся на сервере после сравнения с реально работающим production. Он должен:

- использовать \${API_IMAGE} вместо build: для API;
- сохранить фактический PostgreSQL image, volume/data mount, health check и сеть;
- сохранить фактические имена проекта и сервисов;
- публиковать API только на 127.0.0.1:5001, если Nginx расположен на том же сервере;
- не публиковать PostgreSQL на интерфейс хоста; для связи API достаточно Docker-сети и expose: 5432 либо стандартного внутреннего порта;
- подключать /etc/practicegrading/production.env через параметр --env-file deploy-скрипта.

Изменение Compose-файла само по себе не пересоздаёт контейнеры. Пересоздание происходит только после команды docker compose up. Поэтому конфигурацию можно сначала проверить через docker compose config, не меняя работающий production.

9.3. Nginx

Nginx должен:

- раздавать frontend через стабильный путь /var/www/practice-grading;
- для SPA возвращать index.html на маршрутах, которые не соответствуют физическому файлу;
- проксировать /practice-grading/api в API на 127.0.0.1:5001;
- проксировать SignalR с поддержкой WebSocket;
- учитывать /var/lib/practicegrading-production/maintenance.flag и возвращать 503 во время критической части deploy/rollback.

Точный существующий Nginx-конфиг необходимо получить командой sudo nginx -T, сверить и изменить на месте. Подготовительный скрипт намеренно не угадывает его структуру и не перезаписывает работающую конфигурацию.

10. Алгоритм развёртывания

deployment/deploy-application.sh выполняется от root и берёт настройки из /etc/practicegrading/deployment.env.

1. Захватывает эксклюзивную блокировку flock, исключая параллельный deploy.

2. Проверяет структуру artifact, полный 40-символьный SHA и соответствие образа разрешённому GHCR-репозиторию.
3. Запрещает символические ссылки и неподдерживаемые типы файлов во входных frontend и migrations.
4. Копирует релиз в неизменяемый каталог `/var/lib/practicegrading-production/releases/<SHA>` и назначает безопасные права.
5. Загружает API-образ командой `docker pull` до остановки сервиса.
6. Сохраняет предыдущие `API_IMAGE` и ссылку frontend для возможного rollback.
7. Создаёт `maintenance.flag` и останавливает только API. PostgreSQL продолжает работать.
8. Создаёт custom-format дампа через `pg_dump`, затем проверяет его командой `pg_restore --list`.
9. Применяет ещё не выполненные SQL-миграции в порядке номеров. Контрольная сумма уже применённой миграции не должна изменяться.
10. Атомарно обновляет `API_IMAGE` в защищённом env-файле.
11. Пересоздаёт только API: `docker compose up -d --no-deps <api-service>`.
12. Ожидает успешный GET `/health/live`.
13. Атомарно переключает `/var/www/practice-grading` на frontend нового релиза.
14. Записывает SHA в `current-release` и удаляет `maintenance.flag`.

При самом первом CD существующий каталог frontend переносится в `releases/bootstrap-<timestamp>/frontend`, после чего стабильный путь становится символической ссылкой. Это даёт точку возврата даже для первого автоматического релиза.

11. Автоматический rollback

При ошибке после начала изменяющей части deploy-скрипт запускает обработчик rollback:

1. останавливает API;
2. если миграции могли изменить БД, завершает соединения с базой, пересоздаёт её и восстанавливает проверенный dump;
3. возвращает прежнее значение `API_IMAGE` в protected env;
4. возвращает прежнюю символическую ссылку frontend;
5. пересоздаёт и запускает прежний API;
6. повторяет health check;
7. удаляет maintenance-флаг только после успешного полного восстановления.

Если восстановление завершилось не полностью, maintenance-флаг остаётся на месте, чтобы пользователи не работали с несогласованным состоянием API, frontend и базы. Workflow завершается ошибкой в обоих случаях: даже успешный rollback означает, что новый релиз не был установлен.

Production-подобный тест проверил два сценария в изолированном окружении: позднюю искусственную ошибку после миграций, замены API и переключения frontend с полным возвратом состояния; затем обычный успешный deploy того же релиза.

Автоматический откат действует только во время выполнения скрипта. Ошибка, обнаруженная пользователем после успешного выпуска, не запускает его автоматически. Предыдущий образ сохраняется в Docker, а предыдущий фронтэнд остаётся в каталоге релизов. Скрипт не экспортирует Docker-образы в архив при каждом выпуске и не предоставляет отдельной команды послерелизного отката. Восстановление старой базы после возобновления работы может потерять новые пользовательские изменения и требует отдельного решения администратора.

15. Карта исходных файлов

Файл	Назначение
<code>.github/workflows/ci-cd.yml</code>	CI, упаковка и запуск production deploy
<code>deployment/deploy-application.sh</code>	Транзакционный deploy и rollback
<code>deployment/backup-database.sh</code>	Преддеплойный dump PostgreSQL
<code>deployment/run-migrations.sh</code>	Последовательное применение SQL-миграций
<code>PracticeGrading.Data/SqlMigrations/001_member_access_authentication.sql</code>	Схема заявок, доверенного доступа и изменений пользователей
<code>PracticeGrading.API/Endpoints/UserEndpoints.cs</code>	Вход, пароли, создание администратора, trusted login

Файл	Назначение
PracticeGrading.API/Endpoints/MembersEndpoints.cs	Участники и управление доверенным доступом
PracticeGrading.API/Endpoints/MeetingMemberAccessEndpoints.cs	Заявки, одобрение, отклонение и отзыв
PracticeGrading.API/Auth/ActiveMemberAccessHandler.cs	Проверка, что доступ JWT члена комиссии не отозван
frontend/src/services/ApiService.ts	Клиентские вызовы API и обработка потери авторизации
frontend/src/pages/ProfilePage.tsx	Пароль и регистрация администратора
frontend/src/pages/MemberLoginPage.tsx	Заявка, polling, trusted login и получение JWT
frontend/src/pages/TrustedAccessActivationPage.tsx	Сохранение персонального доверенного токена
frontend/src/components/MemberModal.tsx	Выдача, копирование и отзыв доверенной ссылки
frontend/src/components/MeetingAccessRequestsPanel.tsx	Обработка ожидающих заявок
frontend/src/components/ApprovedMeetingAccessesPanel.tsx	Просмотр и отзыв активных доступов
frontend/tests/access-and-admin.spec.ts	Сквозные тесты новых пользовательских сценариев

16. Pull request

- <https://github.com/yurii-litvinov/PracticeGrading/pull/13>

CD включён и успешно выполнил первый выпуск на рабочем сервере. Проверки ниже фиксируют наблюдавшийся результат, а не гарантируют отсутствие ошибок во всех сценариях.

17. Ежедневное резервное копирование

17.1. Компоненты и настройки

Компонент	Назначение
/usr/local/sbin/practicegrading-database-backup	Исполняемый скрипт создания, проверки и очистки ежедневных копий
/etc/practicegrading/database-backup.env	Настройки, владелец root, права 0600
practicegrading-database-backup.service	Одноразовое задание systemd от root
practicegrading-database-backup.timer	Ежедневный запуск задания
/var/backups/practicegrading/production/daily	Ежедневные дампы и их контрольные суммы

Установленные значения:

```
POSTGRES_CONTAINER=practicegrading-postgres-1
POSTGRES_USER=postgres
POSTGRES_DB=practice_grading
BACKUP_DIR=/var/backups/practicegrading/production/daily
RETENTION_DAYS=14
```

Таймер задаёт OnCalendar=*-*-* 03:15:00, RandomizedDelaySec=15m и Persistent=true. Используется время сервера, при настройке это UTC. Случайная задержка сдвигает запуск в пределах 15 минут, точное ближайшее время показывает `systemctl list-timers`. Пропущенный во время выключения сервера календарный запуск выполняется после активации таймера. Сервис не должен постоянно оставаться активным: после успешного выполнения состояние `inactive (dead)` вместе с `status=0/SUCCESS` нормально.

17.2. Алгоритм

1. Скрипт проверяет настройки, права конфигурации, имя базы и каталог назначения. Устанавливает `umask 077`.
2. Захватывает `flock` на `/run/lock/practicegrading-database-backup.lock`, чтобы не выполнять два ежедневных копирования одновременно. Эта блокировка отдельная от блокировки CD и не синхронизирует копирование с миграциями.
3. В работающем контейнере PostgreSQL запускает `pg_dump` с `--format=custom --no-owner --no-acl`. Вывод записывает во временный файл. Приложение и контейнер базы не останавливаются.
4. Проверяет, что файл не пуст, и выполняет `pg_restore --list`. Это проверка читаемости каталога архива, а не пробное восстановление всех данных.
5. Вычисляет SHA-256 и сохраняет файл `.dump.sha256`. После проверок переименовывает временный дамп в `practice_grading_<UTC timestamp>.dump`, задаёт права 0600.
6. Только после успешного создания новой копии удаляет подходящие ежедневные дампы старше `RETENTION_DAYS` и их файлы контрольных сумм. Удаляет также оставшиеся временные файлы старше суток.

18. Результаты проверки рабочего сервера

10 сентября 2026 года зафиксировано:

- deploy-production завершился с результатом Succeeded.
- API и фронтэнд соответствуют одному релизу, указанному в разделе 1.
- Главная страница отвечает HTTP 200, API /health/live через публичный адрес отвечает HTTP 204, режим обслуживания выключен.
- Миграция 001_member_access_authentication присутствует в __SchemaMigrations, созданы таблицы MeetingMemberAccesses и TrustedMemberAccesses.
- Первый ежедневный дамп practice_grading_20260910T134150Z.dump создан, проверены его SHA-256 и каталог архива. Таймер включён и активен. Полное восстановление именно этого дампа не выполнялось.
- Вручную проверены добавление администратора, смена пароля, подтверждаемый вход, отзыв доступа, активация доверенной ссылки, выставление оценок и генерация документов. Это не исчерпывающая проверка всех интерфейсов и интеграций.

Автоматический откат проверялся в изолированном окружении при искусственной поздней ошибке. На рабочем сервере наблюдался успешный выпуск без ошибки, принудительный откат на проде не запускался.